

Query optimization with triplestores

- Motivation for triplestores from Semantic Web
- Motivation for triplestores from relational database limitations
- Query Language SPARQL
- Query evaluation
- Query cost estimation
- In-memory index join cost
- Join order optimization

Introducing an
example of NoSQL
databases: **triplestores**

The Semantic Web [Lee 1999]

- A web of data.

Tim Berners Lee

In the news

- **Tim Berners-Lee** (*pictured*) wins the Turing Award for inventing the World Wide Web, the first web browser, and the protocols and algorithms that allow the Web to scale.



April 2017

Triplestores: Motivation from Semantic Web

The World Wide Web

- E.g. showtimes page of a movie theatre
- Based on formatted text.

Tivoli Theatre
Mon 7pm showing:
Hi There!

- HTML (Hypertext Markup Language)
- Human readable

The Semantic Web

- Integrate all showtimes from all movie theatres
- Based on semistructured Data: **Triples:**

(“SMo700Tiv”, time, “7 pm”)
(“SMo700Tiv”, movie, “Hi There!”)
... \ /
 Subject Predicate Object

- RDF (Resource Description Format)
- Machine readable

Triplestores: Motivation from Relational Model

Schema:

- `Lecturer(lid, name, room)`
`Course(cid, programme, cnr, year, sem)`
`Teaches(l[lid from Lecturer], c[cid from Course])`

Data:

- `Lecturer(L1234567, Pat, 829)`
`Lecturer(L7654321, Vic, 567)`
`Course(C234567, GENED, 789, 2019, S2)`
`Course(C45678, SOFTENG, 351, 2017, S1)`
`Teaches(L1234567, C234567)`
`Teaches(L7654321, C234567)`
`Teaches(L1234567, C45678)`

Relational data expressed here as facts/ground atoms of predicate calculus

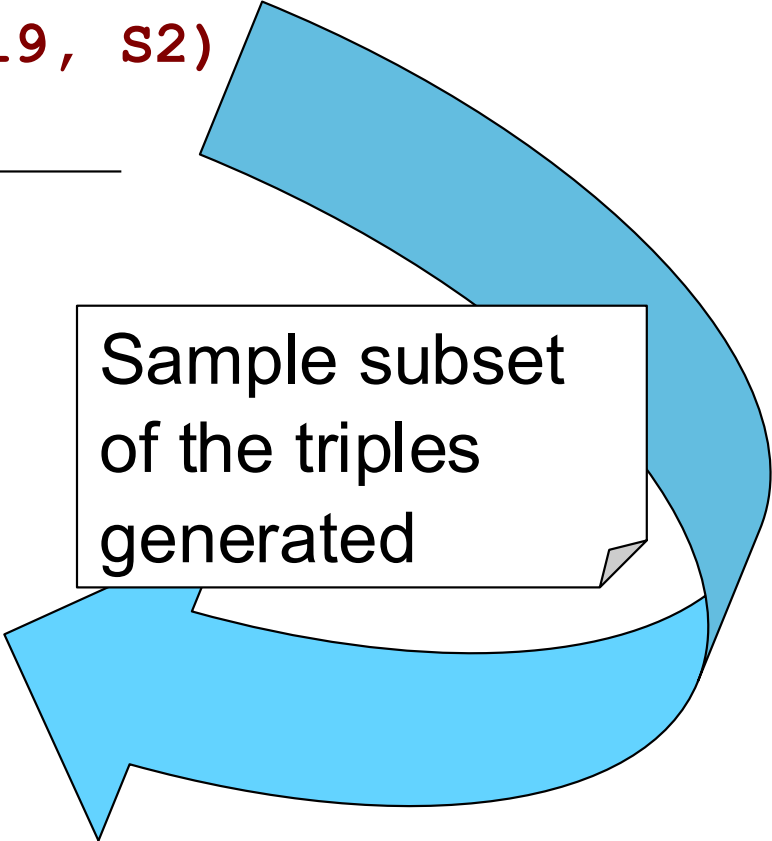
How would we model this if we would have to express all data as triples?

Assume: 1. **keys unique across tables**: no cid clashes with any lid.
2. **all column/table names are unique**

Turning Relational Data into Triples

How would we model the data if we have to use only triples?

- `Lecturer(lid, name, room)`
`Course(cid, programme, cnr, year, sem)`
`Teaches(l[lid from Lecturer], c[cid from Course])`
- `Lecturer(L1234567, Pat, 829)`
`Course(C234567, GENED, 567, 2019, S2)`
`Teaches(L1234567, C234567)`
- `L1234567 name Pat .`
`L7654321 name Vic .`
`L1234567 room 829 .`
`C234567 programme GENED .`
`C45678 programme SOFTENG .`
`C234567 year 2019 .`
`L1234567 teaches C234567 .`

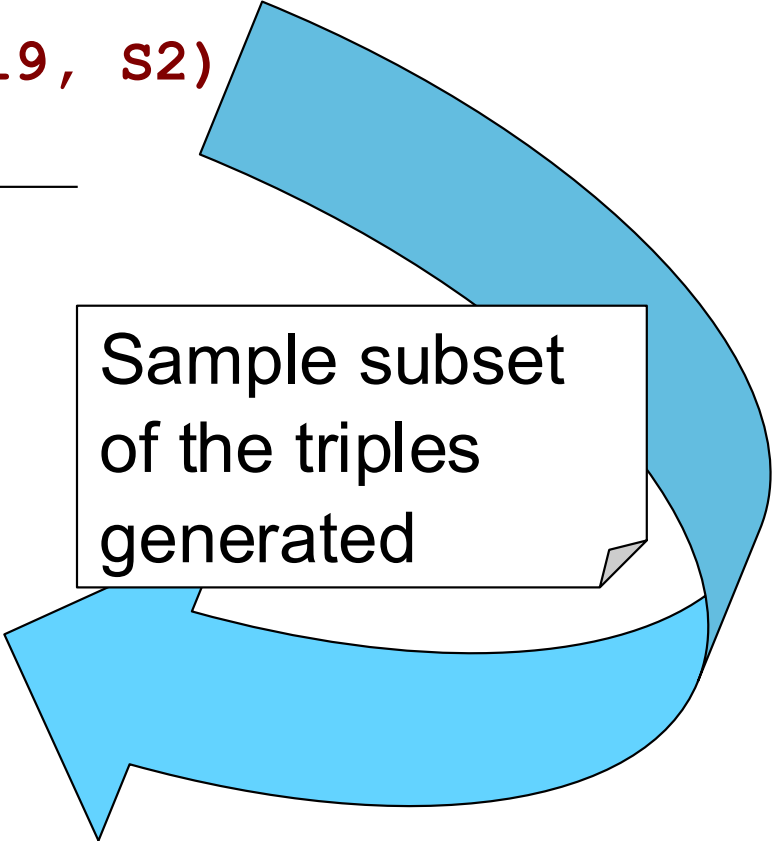


Sample subset
of the triples
generated

Expressing Types

Reserved predicate **a** for expressing isA relation.

- `Lecturer(lid, name, room)`
`Course(cid, programme, cnr, year, sem)`
`Teaches(l[lid from Lecturer], c[cid from Course])`
 - `Lecturer(L1234567, Pat, 829)`
`Course(C234567, GENED, 567, 2019, S2)`
`Teaches(L1234567, C234567)`
-
- `L1234567 name Pat .`
`L1234567 a Lecturer.`
`L1234567 room 829 .`
`C234567 programme GENED .`
`C234567 a Course.`
`L1234567 teaches C234567 .`



Sample subset
of the triples
generated

Advantage of Triples: ad hoc data

Database URI: `http://www.auckland.ac.nz/dbnu`

- `Lecturer(lid, name, room)`
`Lecturer(L1234567, Pat, 829)`
`Teaches(L1234567, C234567)`
- `L1234567 name Pat .`
`L7654321 name Vic .`
`C45678 programme SOFTENG .`
`C234567 year 2019 .`
`L1234567 teaches C234567 .`
- `L1234567 email "pat@akl.ac.nz"`
`L7654321 github vic_akl .`
`L7654321 assesses C45678`

Ad-hoc added
triples,
no need to
define/extend a
relational
schema

Advantage of Triples: ad hoc data

Database URI: `http://www.auckland.ac.nz/dbnu`

- ~~`Lecturer(lid, name, room, github, email)`~~
`Lecturer(L1234567, Pat, 829, NULL, "pat...")`
`Teaches(L1234567, C234567)`

Don't need schema any more.

- `L1234567 name Pat .`
`L7654321 name Vic .`
`C45678 programme SOFTENG .`
`C234567 year 2019 .`
`L1234567 teaches C234567 .`
- `L1234567 email "pat@akl.ac.nz"`
`L7654321 github vic_akl .`
`L7654321 assesses C45678`

Ad-hoc added triples,
no need to
define/extend a
relational
schema

Triplestores and SPARQL semantics

Triplestores

Triplestore: a database to store and query triples such as RDF triples

- Has a single table: **Triple(subject, predicate, object)**
- Should be flexible for collecting data: All three columns are type String
- Can be queried with SPARQL

semistructured

- **L1234567 name Pat .**
L7654321 name Vic .
L1234567 room 829 .
C45678 programme SOFTENG .
C234567 year 2019 .
L1234567 teaches C234567 .
- **L1234567 email "pat@akl.ac.nz" .**
L7654321 github vic_akl .
L7654321 assesses C45678

```
SELECT ?name WHERE {  
  ?lecturer name ?name .  
  ?lecturer teaches ?course1 .  
  ?lecturer teaches ?course2 .  
  ?course1 programme GENED .  
  ?course2 programme  
  SOFTENG .  
}
```

SPARQL

- a recursive acronym for SPARQL Protocol and RDF Query Language
- Idea is following “Query By Example” (QBE) [Zloof 1975]:
- Which data matches the shape of the example=query aka.

basic graph pattern ?

```
?lecturer name ?name .  
?lecturer teaches ?course1 .  
?lecturer teaches ?course2 .  
?course1 programme GENED .  
?course2 programme SOFTENG .
```

```
L1234567 name Pat .  
L7654321 name Vic .  
L1234567 room 829 .  
C234567 programme GENED .  
C45678 programme SOFTENG .  
C234567 year 2019 .  
C45678 year 2017 .  
L1234567 teaches C234567 .  
L1234567 teaches C45678 .  
L7654321 teaches C234567 .  
L7654321 assesses C45678 .
```

- *Triple patterns*: triples with variables:
- *variables* start with a question mark.

SPARQL

- a recursive acronym for SPARQL Protocol and RDF Query Language
- Idea is following “Query By Example” (QBE) [Zloof 1975]: find all *matching subgraphs* for basic graph pattern

- - ?lecturer name ?name .
 - ?lecturer teaches ?course1 .
 - ?lecturer teaches ?course2 .
 - ?course1 programme GENED .
 - ?course2 programme SOFTENG .
- Each Variable in basic graph pattern:
- Single value in each matching subgraph (simply *match*)

→ L1234567 name Pat .
L7654321 name Vic .
L1234567 room 829 .
→ C234567 programme GENED .
→ C45678 programme SOFTENG .
C234567 year 2019 .
C45678 year 2017 .
→ L1234567 teaches C234567 .
→ L1234567 teaches C45678 .
L7654321 teaches C234567 .
L7654321 assesses C45678 .

SPARQL

- **SPARQL** (a recursive acronym for SPARQL Protocol and RDF Query Language)
- SQL like syntax and capabilities (join, filter, aggregate etc).
- Each match gives result row

- ```
SELECT ?name WHERE {
 ?lecturer name ?name .
 ?lecturer teaches ?course1 .
 ?lecturer teaches ?course2 .
 ?course1 programme GENED .
 ?course2 programme SOFTENG .
}
```

- Result set: **Pat**

```
L1234567 name Pat .
L7654321 name Vic .
L1234567 room 829 .
C234567 programme GENED .
C45678 programme SOFTENG .
C234567 year 2019 .
C45678 year 2017 .
L1234567 teaches C234567 .
L1234567 teaches C45678 .
L7654321 teaches C234567 .
L7654321 assesses C45678 .
```

# SPARQL: variable predicates

- **SPARQL** allows generally variables in predicate position:
- Some databases may only allow SPARQL queries with no variables in predicate position (est. 77% of queries)
- ```
SELECT ?name, ?has WHERE {  
  ?lecturer name ?name .  
  ?lecturer teaches ?course1 .  
  ?lecturer ?has ?course2 .  
  ?course1 programme GENED .  
  ?course2 programme SOFTENG .  
}
```

```
L1234567 name Pat .  
L7654321 name Vic .  
L1234567 room 829 .  
C234567 programme GENED .  
C45678 programme SOFTENG .  
C234567 year 2019 .  
C45678 year 2017 .  
L1234567 teaches C234567 .  
L1234567 teaches C45678 .  
L7654321 teaches C234567 .  
L7654321 assesses C45678 .
```
- Result set?: Pat, teaches
- ???

SPARQL has bag semantics as usual for languages in practice

SPARQL: variable predicates

- **SPARQL** allows generally variables in predicate position:
- Some databases may only allow SPARQL queries with no variables in predicate position (est. 77% of queries).
- ```
SELECT ?name, ?has WHERE {
 ?lecturer name ?name .
 ?lecturer teaches ?course1 .
 ?lecturer ?has ?course2 .
 ?course1 programme GENED .
 ?course2 programme SOFTENG .
}
```
- Result set: Pat, teaches;  
Vic, assesses

```
L1234567 name Pat .
L7654321 name Vic .
L1234567 room 829 .
C234567 programme GENED .
C45678 programme SOFTENG .
C234567 year 2019 .
C45678 year 2017 .
L1234567 teaches C234567 .
L1234567 teaches C45678 .
L7654321 teaches C234567 .
L7654321 assesses C45678 .
```

SPARQL has bag semantics as usual for languages in practice

# SPARQL tip: think in terms of matches

- Projection by SELECT is an afterthought.

- ```
SELECT ?name, ?has WHERE {  
  ?lecturer name ?name .  
  ?lecturer teaches ?course1 .  
  ?lecturer ?has ?course2 .  
  ?course1 programme GENED .  
  ?course2 programme SOFTENG .  
}
```

- Result set: Pat, teaches;
- Vic, assesses

```
L1234567 name Pat .  
C234567 programme GENED .  
C45678 programme SOFTENG .  
L1234567 teaches C234567 .  
L1234567 teaches C45678 .
```

```
L7654321 name Vic .  
C234567 programme GENED .  
C45678 programme SOFTENG .  
L7654321 teaches C234567 .  
L7654321 assesses C45678 .
```

SPARQL: filters

- Filters remove matches that evaluate the filter to false.
- Filters allow common Boolean expressions

- `SELECT ?name, ?has WHERE {
 ?lecturer name ?name .
 ?lecturer teaches ?course1 .
 ?lecturer ?has ?course2 .
 ?lecturer room ?room .
 ?course1 programme GENED .
 ?course2 programme SOFTENG .
 FILTER(?room<600)
}`

- Result Set: Vic, assesses

```
L1234567 name Pat .  
L7654321 name Vic .  
L1234567 room 829 .  
L7654321 room 567 .  
C234567 programme GENED .  
C45678 programme SOFTENG .  
C234567 year 2019 .  
C45678 year 2017 .  
L1234567 teaches C234567 .  
L1234567 teaches C45678 .  
L7654321 teaches C234567 .  
L7654321 assesses C45678 .
```

SPARQL: filters

- Filter expressions allowed:
- Comparison operators: =, !=, >, <, ...
- Logical Operators: !, &&, ||
- Arithmetical operators +, -, *, /
- Parentheses: allowing nested expressions
- **`FILTER NOT EXISTS { ?person foaf:name ?name }`**

Globally Unique Identifiers as URIs

Database URI: `http://www.auckland.ac.nz\dbnu`

System URI: `http://www.nusoft.com\dbnu`

- `Lecturer(lid, Name, Room)`
`Lecturer(L1234567, Pat, 829)`

Use URIs to make identifiers globally unique:

- `www.auckland.ac.nz\dbnu\L1234567`
`www.nusoft.com\dbnu\teaches`

Use aliases to avoid repeating long prefixes:

- PREFIX uoa: `<http://www.auckland.ac.nz\dbnu/>`
- PREFIX dbnu: `<http://www.nusoft.com\dbnu/>`
- `uoal:L1234567 dbnu:name Pat .`
- `uoal:L1234567 dbnu:teaches uoa:C234567 .`

Example SPARQL Query with prefixes

- A SPARQL query with prefixes:

- PREFIX uoa: <http://www.auckland.ac.nz/dbnu/db>

- PREFIX dbnu: <http://www.nusoft.com/dbnu/>

- SELECT ?name, ?has WHERE {

- ?lecturer dbnu:name ?name .

- ?lecturer dbnu:teaches ?course1 .

- ?lecturer ?has ?course2 .

- ?course1 dbnu:programme

- dbnu:GENED .

- ?course2 dbnu:programme

- uoa:SOFTENG .

- }

```
L1234567 dbnu:name Pat .
L7654321 dbnu:name Vic .
L1234567 dbnu:room 829 .
C234567 dbnu:programme dbnu:GENED .
C45678 dbnu:programme uoa:SOFTENG .
C43434 dbnu:programme uoa:MATH .
C234567 dbnu:year 2019 .
C45678 dbnu:year 2017 .
L1234567 dbnu:teaches C234567 .
L1234567 dbnu:teaches C45678 .
L7654321 dbnu:teaches C234567 .
L7654321 dbnu:assesses C45678 .
```

SPARQL: Naïve Query evaluation

1. Find all candidate matches for the graph pattern.
2. Evaluate filters, reject candidate match if false.

- `SELECT ?name, ?has WHERE {
 ?lecturer name ?name .
 ?lecturer teaches ?course1 .
 ?lecturer ?has ?course2 .
 ?lecturer room ?room .
 ?course1 programme GENED .
 ?course2 programme SOFTENG .
 FILTER(?room<600)
}`

```
L1234567 name Pat .  
L7654321 name Vic .  
L1234567 room 829 .  
L7654321 room 567 .  
C234567 programme GENED .  
C45678 programme SOFTENG .  
C234567 year 2019 .  
C45678 year 2017 .  
L1234567 teaches C234567 .  
L1234567 teaches C45678 .  
L7654321 teaches C234567 .  
L7654321 assesses C45678 .
```

3. Result Set: projection by
SELECT clause.

Summary

Triplestores

- Relational data may be stored in triples.
- In triplestores, ad-hoc predicates can be used to add more data
- Can be queried by SPARQL

SPARQL

- Is a query-by example approach.
- Allows to query with variable predicates:
 - a semistructured feature that goes beyond SQL